

Probabilistic Programming

Marius Popescu

popescunmarius@gmail.com

2025 - 2026

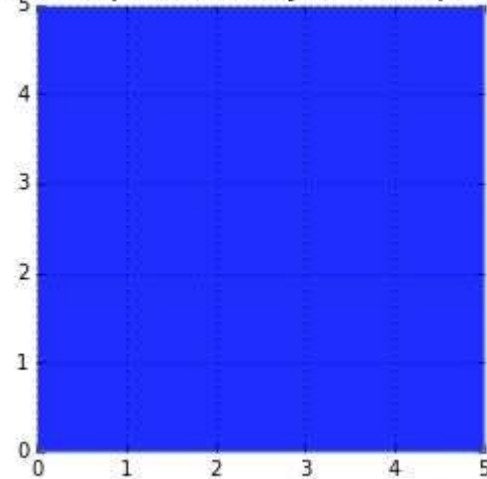
Fitting Models

Markov Chain Monte Carlo (MCMC)

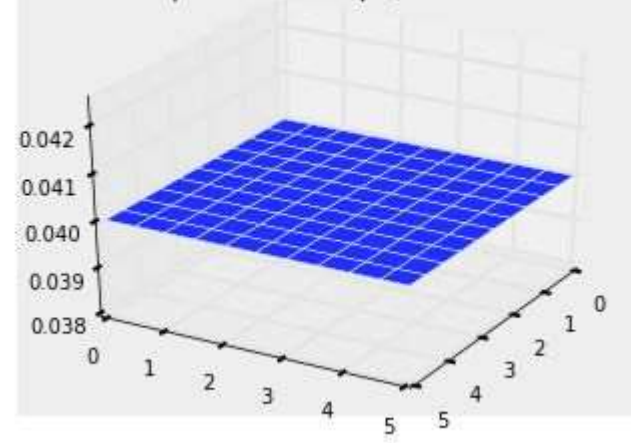
The Bayesian Landscape

When we setup a Bayesian inference problem with N unknowns, we are implicitly creating an N dimensional space for the prior distributions to exist in. Associated with the space is an additional dimension, which we can describe as the surface, or curve, that sits on top of the space, that reflects the prior probability of a particular point. The surface on the space is defined by our prior distributions. For example, if we have two unknowns p_1 and p_2 , and priors for both are $\text{Uniform}(0,5)$, the space created is a square of length 5 and the surface is a flat plane that sits on top of the square (representing that every point is equally likely):

Landscape formed by Uniform priors.



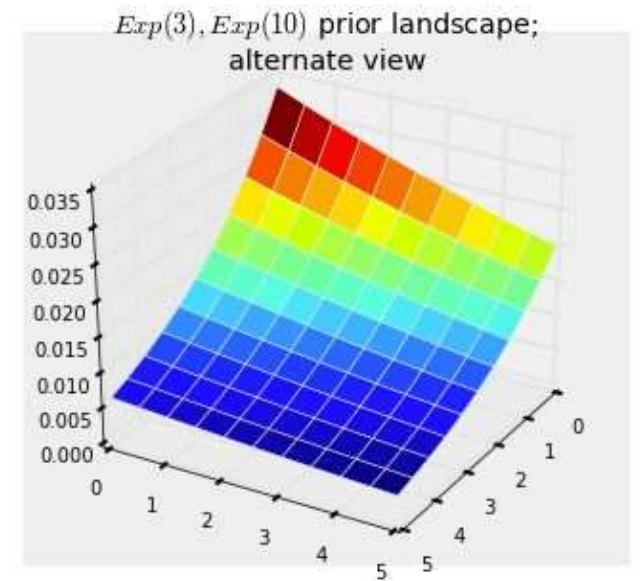
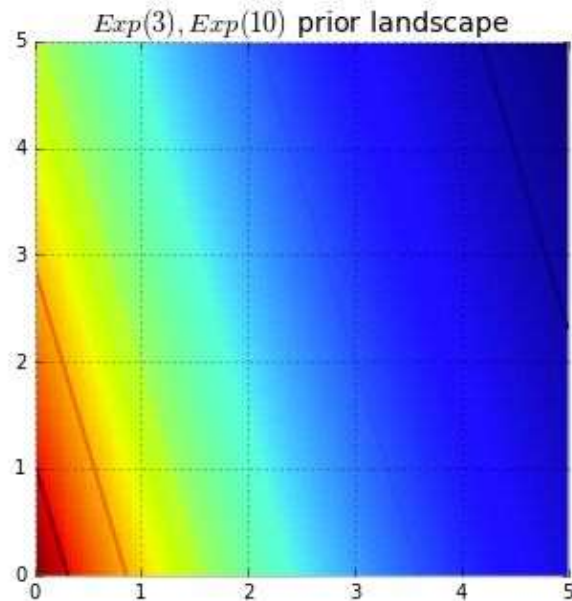
Uniform prior landscape; alternate view



The Bayesian Landscape

Alternatively, if the two priors are $\text{Exp}(3)$ and $\text{Exp}(10)$, then the space is all positive numbers on the 2-D plane, and the surface induced by the priors looks like a water fall that starts at the point $(0,0)$ and flows over the positive numbers.

The plots below visualize this. The more dark red the color, the more prior probability is assigned to that location. Conversely, areas with darker blue represent that our priors assign very low probability to that location:



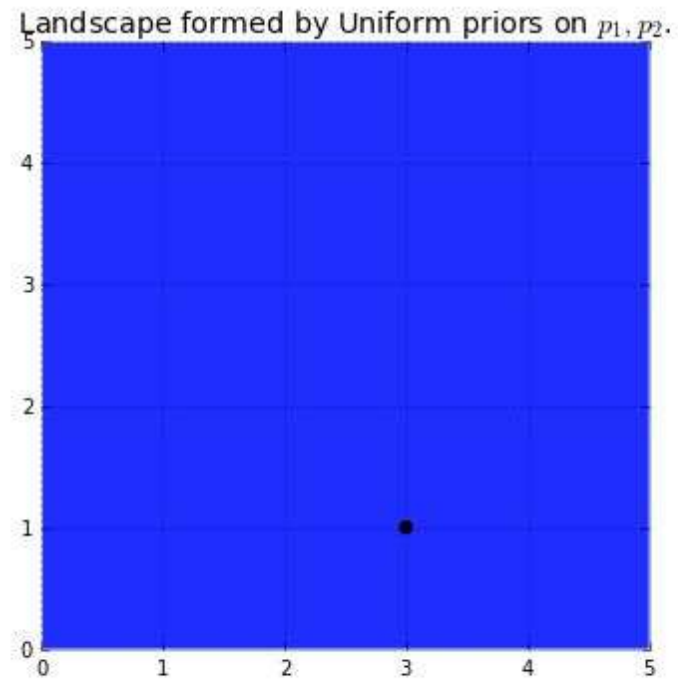
The Bayesian Landscape

If these surfaces describe our *prior distributions* on the unknowns, what happens to our space after we incorporate our observed data X ? The data X does not change the space, but it changes the surface of the space by *pulling and stretching the fabric of the prior surface* to reflect where the true parameters likely live. More data means more pulling and stretching, and our original shape becomes mangled or insignificant compared to the newly formed shape. Less data, and our original shape is more present. Regardless, the resulting surface describes the *posterior distribution*.

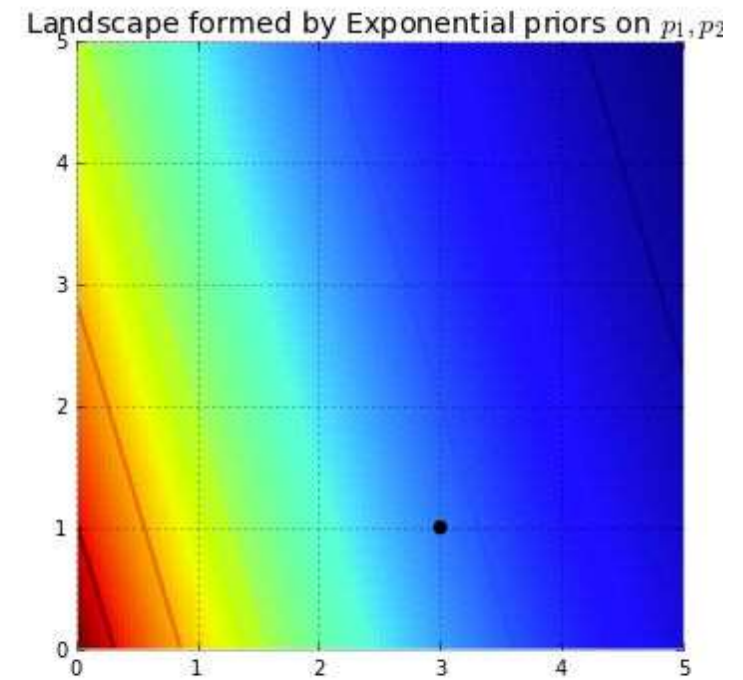
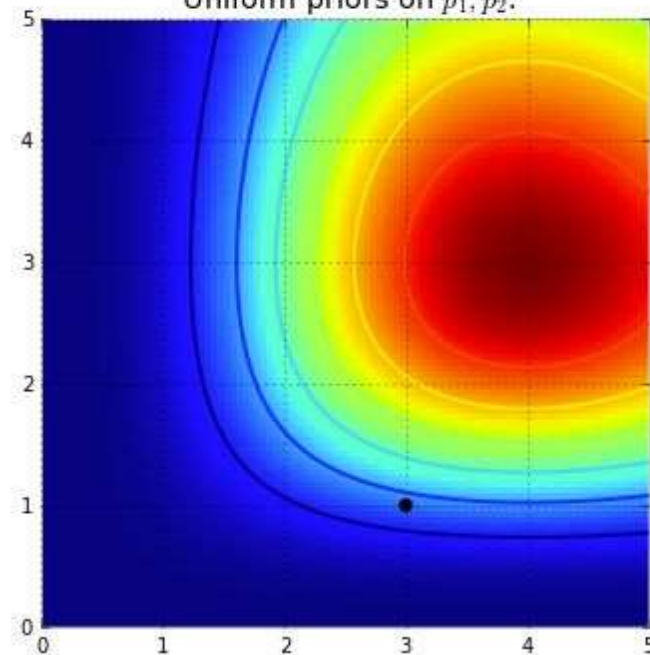
For two dimensions, the data essentially *pushes up* the original surface to make *tall mountains*. The tendency of the observed data to *push up* the posterior probability in certain areas is checked by the prior probability distribution, so that lower prior probability means more resistance. Thus in the double-exponential prior case above, a mountain (or multiple mountains) that might erupt near the (0,0) corner would be much higher than mountains that erupt closer to (5,5), since there is more resistance (low prior probability) near (5,5). The peak reflects the posterior probability of where the true parameters are likely to be found. Importantly, if the prior has assigned a probability of 0, then no posterior probability will be assigned there.

The Bayesian Landscape

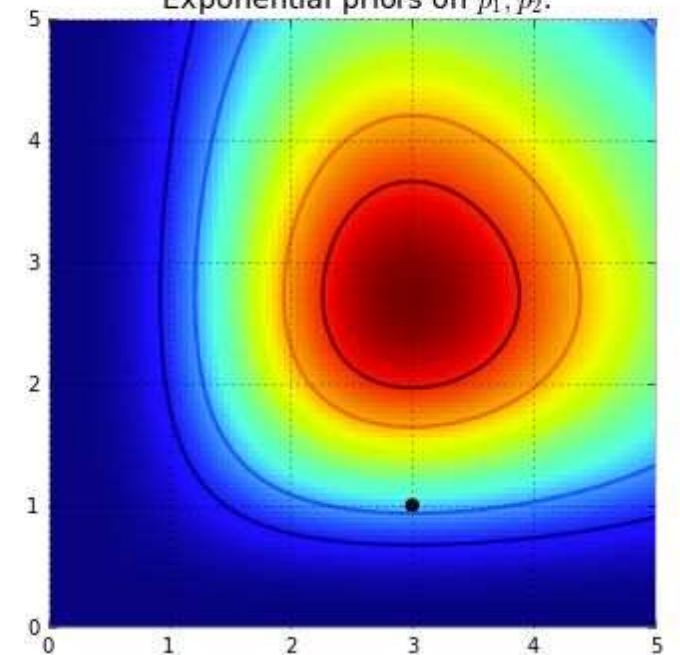
Suppose the priors mentioned above represent different parameters λ of two Poisson distributions. We observe a few data points and visualize the new landscape:



Landscape warped by 1 data observation;
Uniform priors on p_1, p_2 .



Landscape warped by 1 data observation;
Exponential priors on p_1, p_2 .



Exploring the Landscape using the MCMC

We should explore the deformed posterior space generated by our prior surface and observed data to find the posterior mountain.

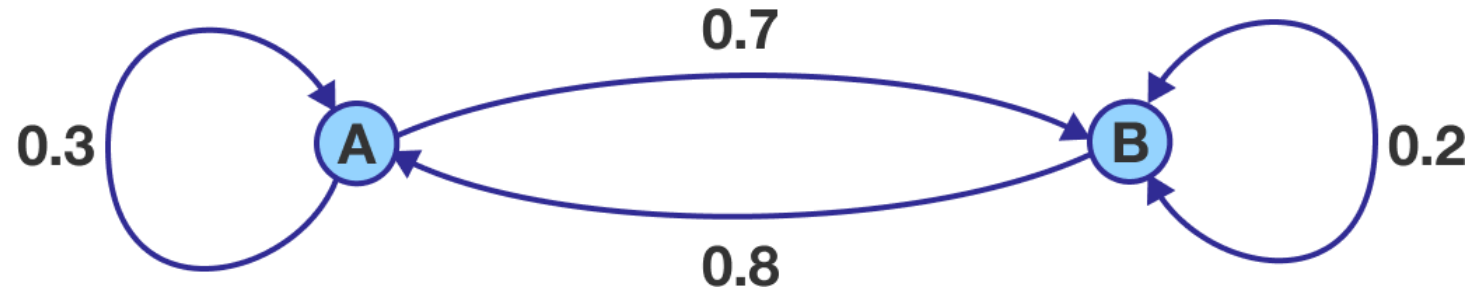
The idea behind MCMC is to perform an intelligent search of the space. To say "search" implies we are looking for a particular point, which is perhaps not accurate as we are really looking for a broad mountain.

MCMC returns samples from the posterior distribution, not the distribution itself. Stretching our mountainous analogy to its limit, MCMC performs a task similar to repeatedly asking "How likely is this pebble I found to be from the mountain I am searching for?", and completes its task by returning thousands of accepted pebbles in hopes of reconstructing the original mountain.

Exploring the Landscape using the MCMC

1. Start at current position.
2. Propose moving to a new position (investigate a pebble near you).
3. Accept/Reject the new position based on the position's adherence to the data and prior distributions (ask if the pebble likely came from the mountain).
4. If you accept: Move to the new position. Return to Step 1.
Else: Do not move to new position. Return to Step 1.
1. After a large number of iterations, return all accepted positions.

Markov Chains



$X_t \in \{A, B\}$, $\{A, B\}$ state space (A, B states)

π_0 the distribution of X_0 (initial distribution of states) $\pi_0 = \begin{pmatrix} A & B \\ 0.5 & 0.5 \end{pmatrix}$

$P = \begin{matrix} & \begin{matrix} A & B \end{matrix} \\ \begin{matrix} A \\ B \end{matrix} & \begin{pmatrix} 0.3 & 0.7 \\ 0.8 & 0.2 \end{pmatrix} \end{matrix}$ transition matrix

$$\pi_1 = \pi_0 P = (0.5 \ 0.5) \begin{pmatrix} 0.3 & 0.7 \\ 0.8 & 0.2 \end{pmatrix} = (0.55 \ 0.45), \quad \pi_2 = \pi_1 P, \quad \dots$$

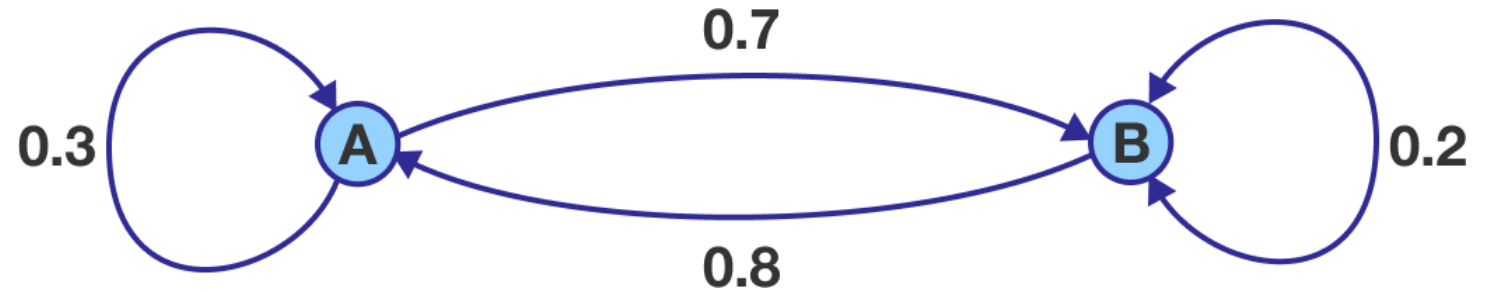
A Markov chain is a special type of *stochastic process*. The standard definition of a stochastic process is an ordered collection of random variables: $\{X_t : t \in T\}$ where t is frequently (but not necessarily) a time index.

If we think of X_t as a state X at time t , and invoke the following dependence condition on each state:

$$P(X_{t+1} = x_{t+1} | X_t = x_t, X_{t-1} = x_{t-1}, \dots, X_0 = x_0) = P(X_{t+1} = x_{t+1} | X_t = x_t)$$

then the stochastic process is known as a Markov chain. This conditioning specifies that the future depends on the current state, but not past states. Thus, the Markov chain wanders about the state space, remembering only where it has just been in the last time step. The collection of transition probabilities is sometimes called a *transition matrix* when dealing with discrete states, or more generally, a *transition kernel*.

Markov Chains



$$\pi = \begin{pmatrix} \frac{8}{15} & \frac{7}{15} \end{pmatrix}$$

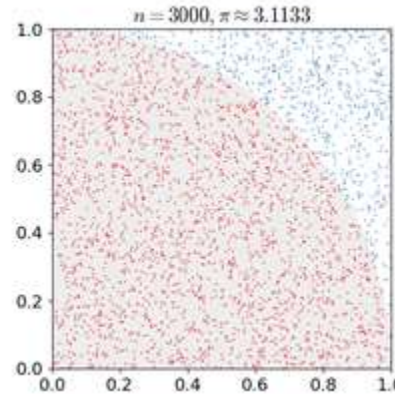
$$\begin{pmatrix} \frac{8}{15} & \frac{7}{15} \end{pmatrix} \begin{pmatrix} \frac{3}{10} & \frac{7}{10} \\ \frac{8}{10} & \frac{2}{10} \end{pmatrix} = \begin{pmatrix} \frac{8}{15} & \frac{7}{15} \end{pmatrix}$$

A *stationary distribution* of a Markov chain is a probability distribution that remains unchanged in the Markov chain as time progresses:

$$\pi = \pi P$$

Ergodic Markov chains have a unique stationary distribution

Monte Carlo Method



Monte Carlo method applied to approximating the value of π . After placing 30,000 random points, the estimate for π is within 0.07% of the actual value.

Monte Carlo methods are a broad class of computational algorithms that rely on repeated random sampling to obtain numerical results. Their essential idea is using randomness to solve problems that might be deterministic in principle

$$I = \int h(x)f(x)dx \quad f \text{ a probability density function}$$

If we can produce a reasonable number of random vectors x_i sampled from f we can use these values to approximate the unknown integral by finite sums:

$$\hat{I} = \frac{1}{n} \sum_{i=1}^n h(x_i)$$

By the strong law of large numbers: $\hat{I} \rightarrow I$ with probability 1

MCMC: A Class of Algorithms

- Design a Markov chain that has the desired distribution (posterior distribution) as its stationary distribution (step function / method)
- Simulate (Monte Carlo) the Markov chain a sufficient number of steps to reach its stationary distribution (burn-in)
- Simulate the Markov chain to sample from the desired distribution (posterior distribution)

The Metropolis-Hastings Algorithm

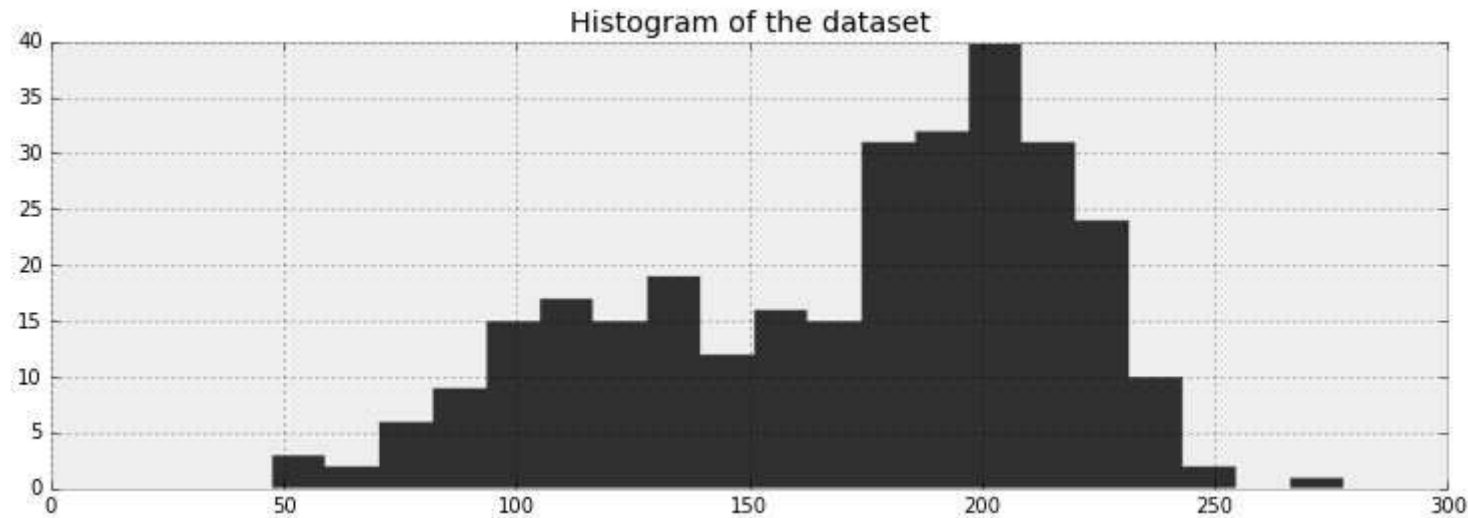
- The Metropolis-Hastings algorithm generates candidate state transitions from an alternate distribution, and accepts or rejects each candidate probabilistically.
- A proposal distribution $Q(\theta'|\theta)$ will be used to propose the next candidate θ' from the current state θ . The proposal distribution must be able to explore the entire space.
- The proposed parameter value θ' is accepted with some probability given by the acceptance probability $\alpha(\theta', \theta) = \min(1, \frac{\pi(\theta')q(\theta|\theta')}{\pi(\theta)q(\theta'|\theta)})$. If the proposal distribution is symmetric (as originally proposed by Metropolis) then: $\alpha(\theta', \theta) = \min(1, \frac{\pi(\theta')}{\pi(\theta)})$.
- The chain is generated from the proposed parameter θ' as follow

$$\theta^{t+1} = \begin{cases} \theta' & \text{if accepted; } \alpha(\theta', \theta) > u \text{ } u \sim \text{Uniform}(0,1) \\ \theta^t & \text{otherwise} \end{cases}$$

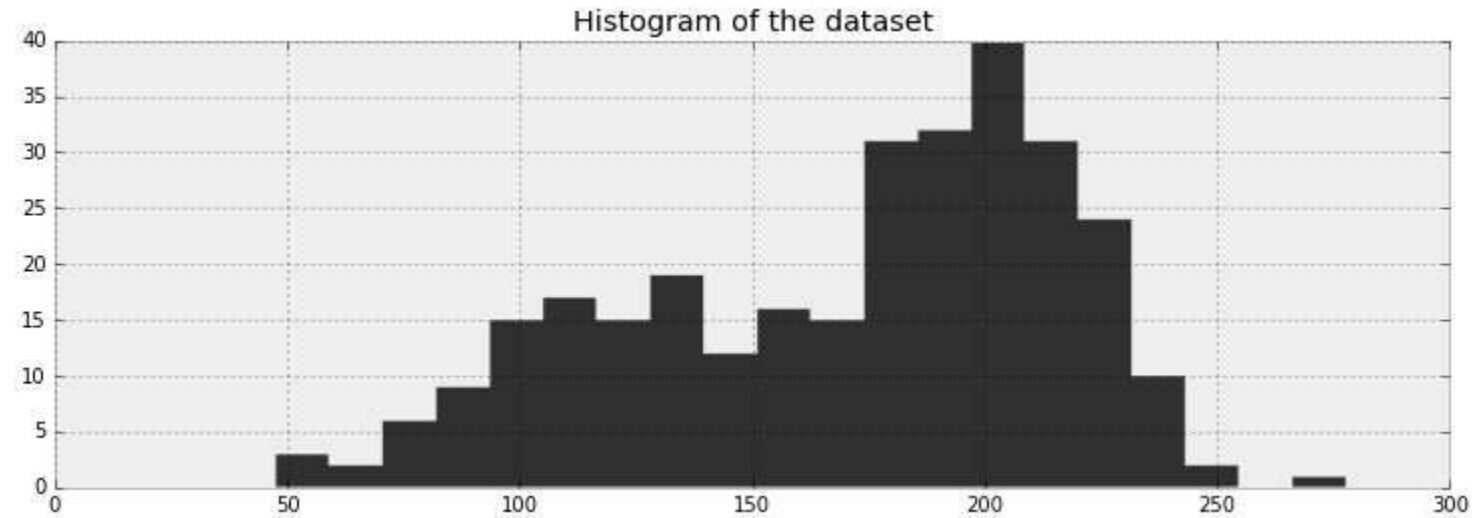
MCMC in PyMC

Example: Unsupervised Clustering using a Mixture Model

Suppose we are given the following dataset. What does the data suggest?



Mixture Model



It appears the data has a bimodal form, that is, it appears to have two peaks, one near 120 and the other near 200. Perhaps there are two clusters within this dataset.

We can propose how the data might have been created:

1. For each data point, choose cluster 1 with probability p , else choose cluster 2.
2. Draw a random variate from a Normal distribution with parameters μ_i and σ_i where i was chosen in step 1.
3. Repeat.

This algorithm would create a similar effect as the observed dataset, so we choose this as our model. Of course, we do not know p or the parameters of the Normal distributions. Hence we must infer, or learn, these unknowns.

PyMC Mixture Model

Denote the Normal distributions N_0 and N_1 (having variables' index start at 0 is just Pythonic). Both currently have unknown mean and standard deviation, denoted μ_i and $\sigma_i, i = 0, 1$ respectively. A specific data point can be from either N_0 or N_1 , and we assume that the data point is assigned to N_0 with probability p .

```
data = np.loadtxt("mixture_data.csv", delimiter=",")
```

```
with pm.Model() as model:
```

```
    p1 = pm.Uniform('p', 0, 1)
```

```
    p2 = 1 - p1
```

```
    p = pt.stack([p1, p2])
```

```
    assignment = pm.Categorical("assignment", p,  
                                shape=data.shape[0],  
                                initval=np.random.randint(0, 2, data.shape[0]))
```

```
with model:
```

```
    sds = pm.Uniform("sds", 0, 100, shape=2)
```

```
    # form new API, may use the sigma/tau for parameter tau = 1/(sigma**2),
```

```
    # the sd is not supported anymore in Normal
```

```
    # https://www.pymc.io/projects/docs/en/stable/api/distributions/generated/pymc.Normal.html
```

```
    centers = pm.Normal("centers",  
                        mu=np.array([120, 190]),  
                        sigma=np.array([10., 10.]),  
                        shape=2)
```

```
    center_i = pm.Deterministic('center_i', centers[assignment])
```

```
    sd_i = pm.Deterministic('sd_i', sds[assignment])
```

```
# and to combine it with the observations:
```

```
observations = pm.Normal("obs", mu=center_i, sigma=sd_i, observed=data)
```

MCMC Sampling

Any sampling that we do within the context of `Model()` will be done only on the model whose context in which we are working. We will tell our model to explore the space that we have so far defined by defining the sampling methods, in this case `Metropolis()` for our continuous variables and `CategoricalGibbsMetropolis()` for our categorical variable. We will use these sampling methods together to explore the space by using `sample( iterations, step )`, where `iterations` is the number of steps you wish the algorithm to perform and `step` is the way in which you want to handle those steps. We use our combination of `Metropolis()` and `CategoricalGibbsMetropolis ()` for the step and sample 25000 iterations below:

with `model`:

```
# We use the CategoricalGibbsMetropolis, and return_inferencedata=False here for compatibility.
step1 = pm.Metropolis(vars=[p1,sds, centers])
step2 = pm.CategoricalGibbsMetropolis(vars=[assignment])
trace1 = pm.sample(25000, step=[step1, step2],return_inferencedata=False,tune=5000,chains=1)
```

Plotting Traces

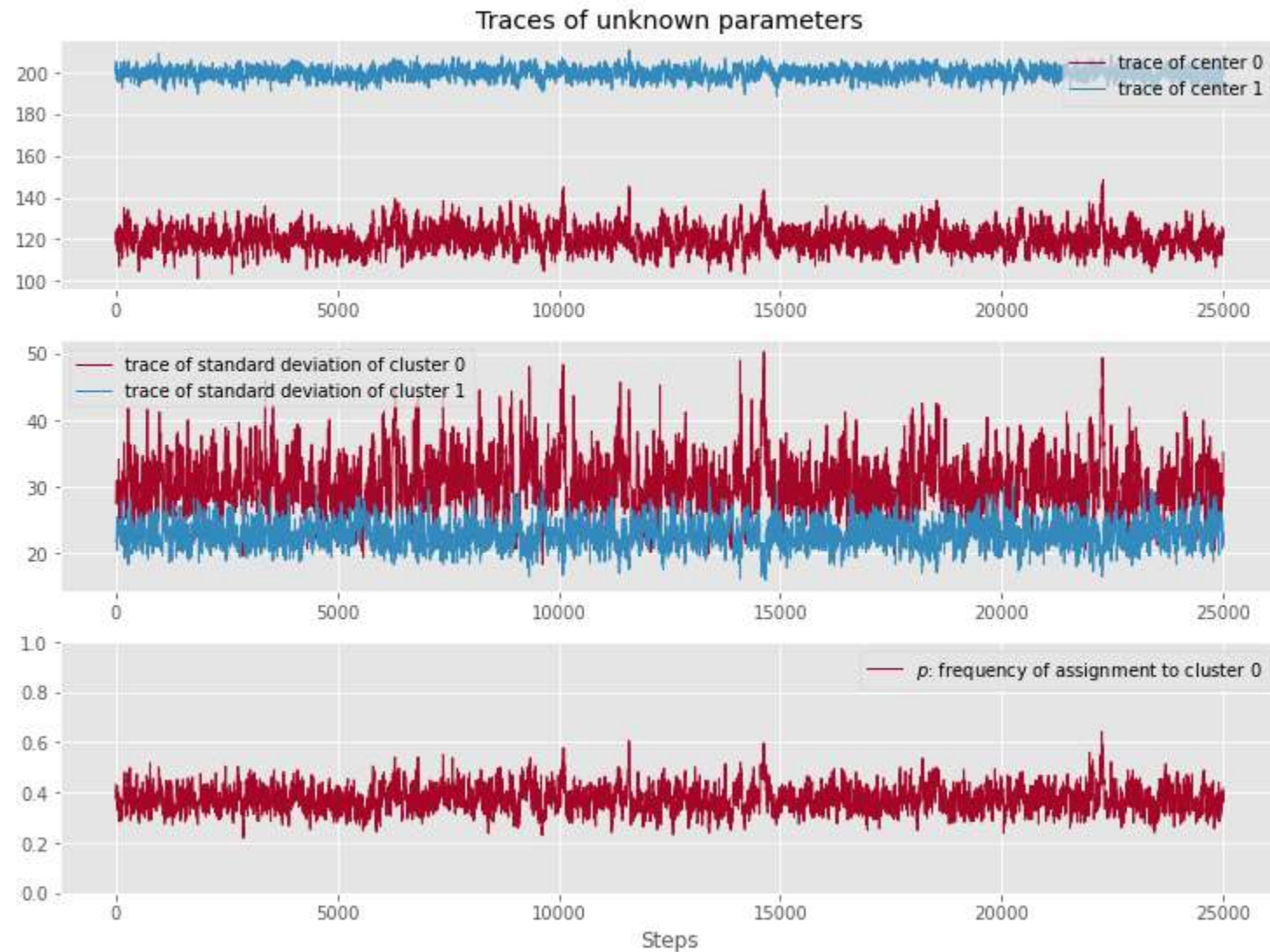
```
plt.subplot(311)
lw = 1
center_trace = trace1["centers"]

# for pretty colors later in the book.
colors = ["#348ABD", "#A60628"] if center_trace[-1, 0] > center_trace[-1, 1] \
    else ["#A60628", "#348ABD"]

plt.plot(center_trace[:, 0], label="trace of center 0", c=colors[0], lw=lw)
plt.plot(center_trace[:, 1], label="trace of center 1", c=colors[1], lw=lw)
plt.title("Traces of unknown parameters")
leg = plt.legend(loc="upper right")
leg.get_frame().set_alpha(0.7)

plt.subplot(312)
std_trace = trace1["sds"]
plt.plot(std_trace[:, 0], label="trace of standard deviation of cluster 0", c=colors[0], lw=lw)
plt.plot(std_trace[:, 1], label="trace of standard deviation of cluster 1", c=colors[1], lw=lw)
plt.legend(loc="upper left")

plt.subplot(313)
p_trace = trace1["p"]
plt.plot(p_trace, label="$p$: frequency of assignment to cluster 0", color=colors[0], lw=lw)
plt.xlabel("Steps")
plt.ylim(0, 1)
plt.legend()
plt.show()
```



Notice the following characteristics:

1. The traces converge, not to a single point, but to a distribution of possible points. This is convergence in an MCMC algorithm.
2. Inference using the first few thousand points is a bad idea, as they are unrelated to the final distribution we are interested in. Thus it is a good idea to discard those samples before using the samples for inference. We call this period before convergence the burn-in period. You can use the parameter `tune` and `discard_tuned_samples` (True by default) to do the burn-in period.
3. The traces appear as a random "walk" around the space, that is, the paths exhibit correlation with previous positions. This is both good and bad. We will always have correlation between current positions and the previous positions, but too much of it means we are not exploring the space well. This will be detailed later.

Hamiltonian Monte Carlo and No-U-Turn Sampler

PyMC also features next-generation Markov chain Monte Carlo (MCMC) sampling algorithms such as the No-U-Turn Sampler (NUTS; Hoffman, 2014), a self-tuning variant of Hamiltonian Monte Carlo (HMC; Duane, 1987). This class of samplers works well on high-dimensional and complex posterior distributions and allows many complex models to be fit without specialized knowledge about fitting algorithms. HMC and NUTS take advantage of gradient information from the likelihood to achieve much faster convergence than traditional sampling methods, especially for larger models. NUTS also has several self-tuning strategies for adaptively setting the tunable parameters of Hamiltonian Monte Carlo, which means you usually don't need to have specialized knowledge about how the algorithms work.

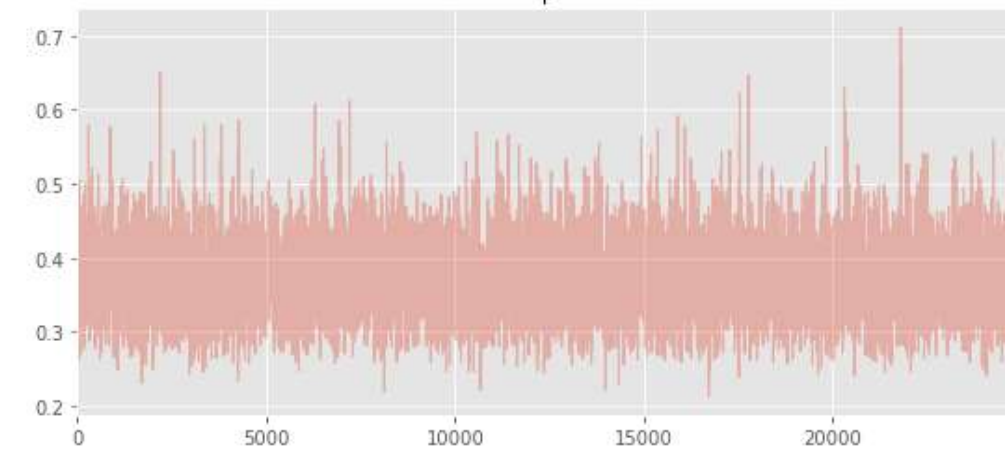
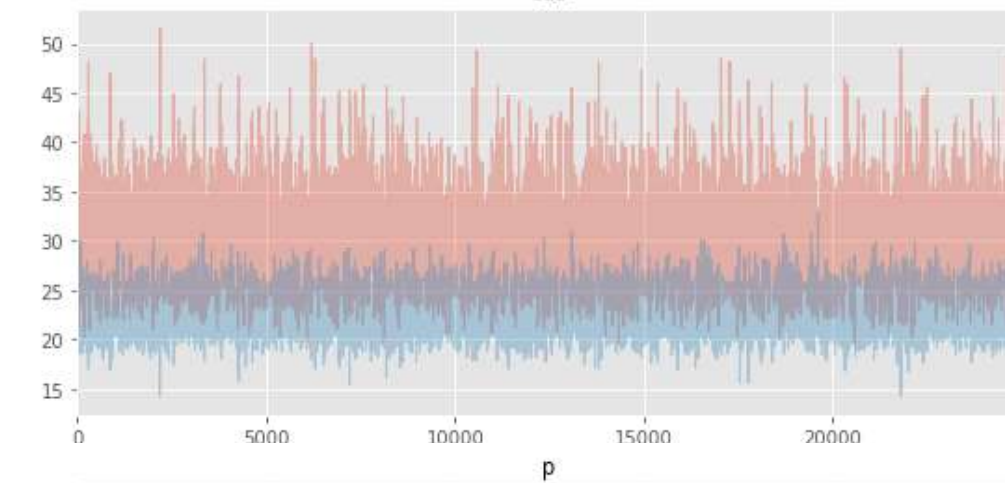
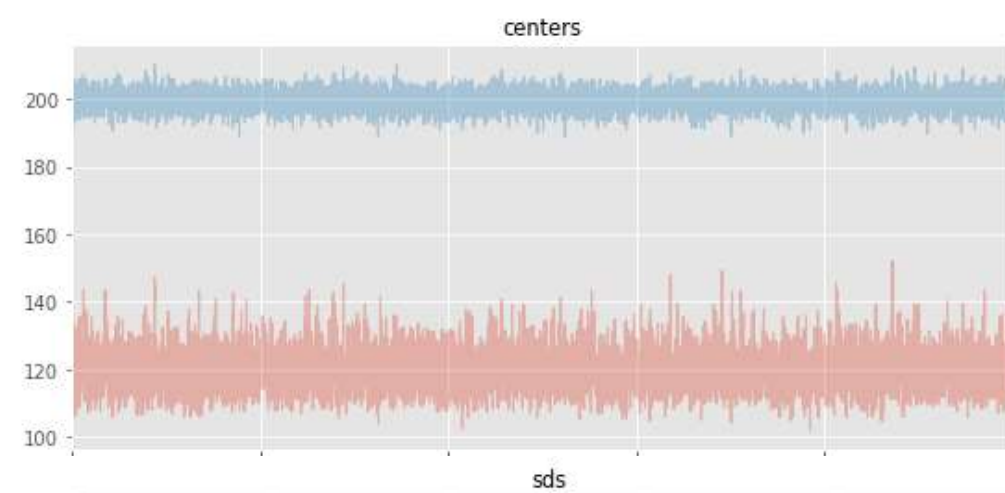
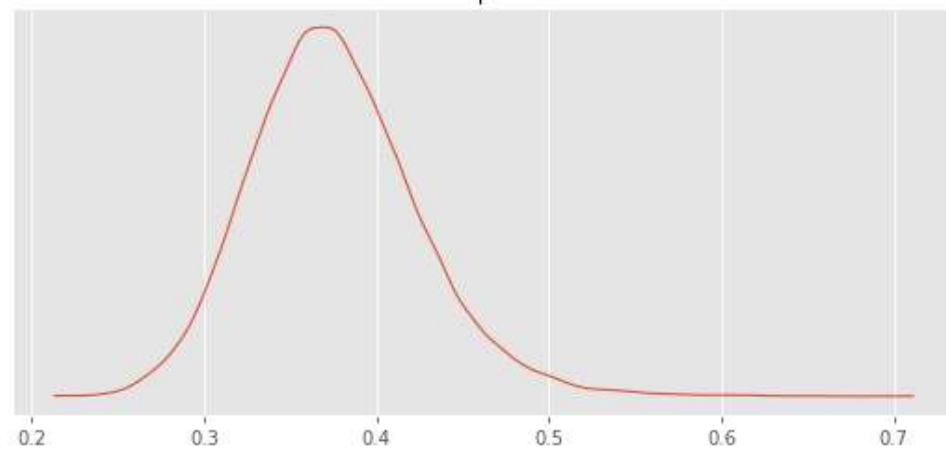
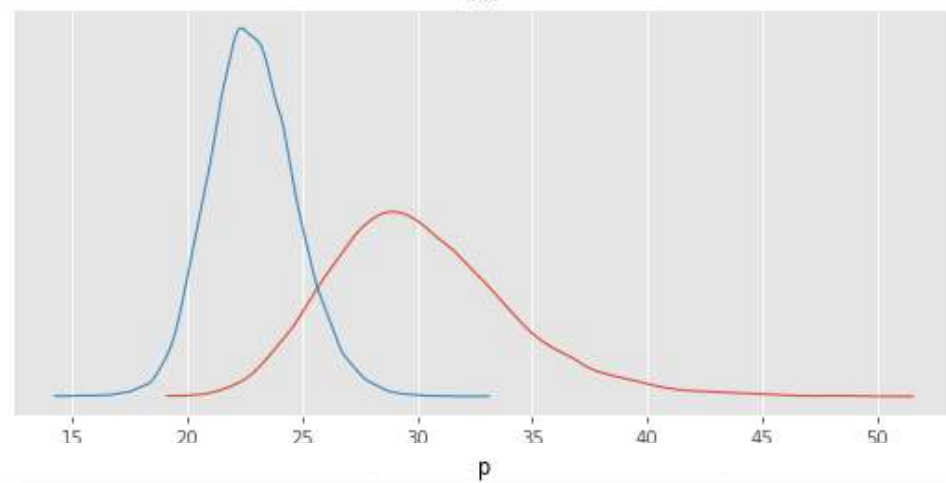
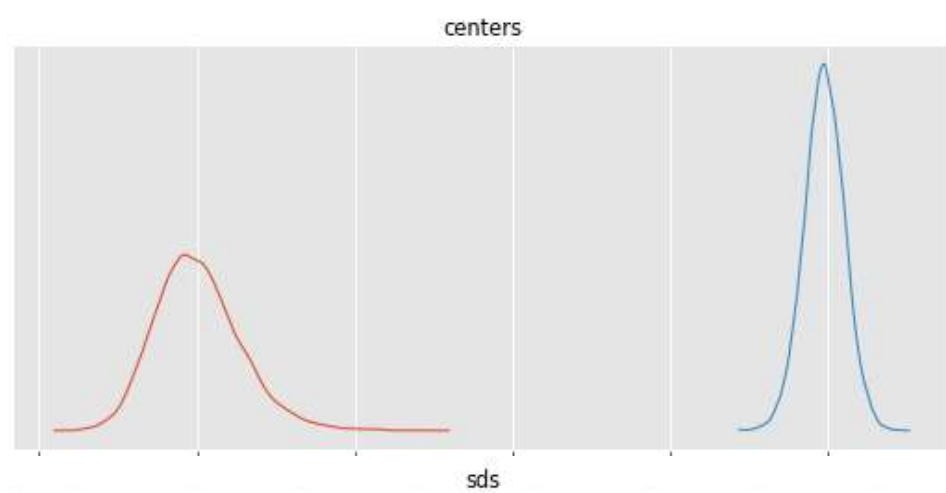
PyMC: NUTS and Arviz

with model:

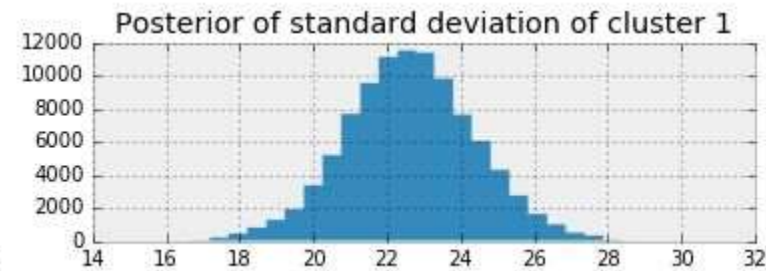
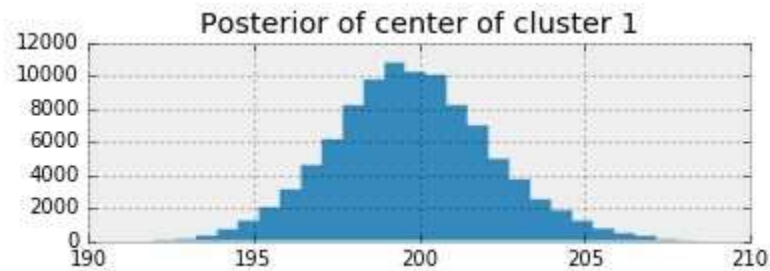
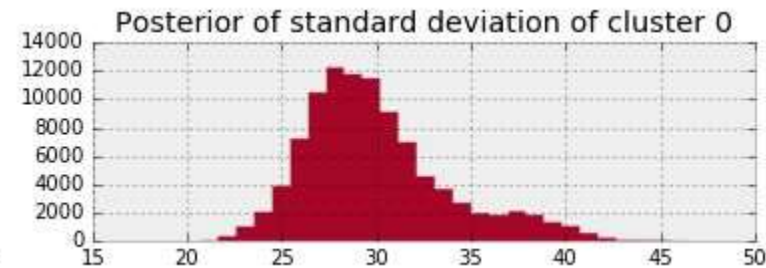
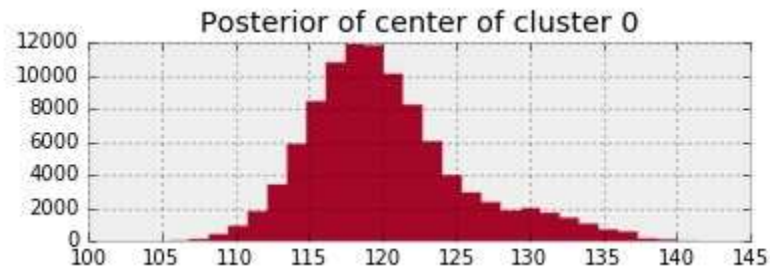
```
# Use the default NUTS for sampling, and we return the Arviz InferenceData
step3 = pm.CategoricalGibbsMetropolis(vars=[assignment])
trace2 = pm.sample(25000, step=[step3], tune=5000, chains=1)
```

The powerful Arviz InferenceData and APIs give us more comfortable view:

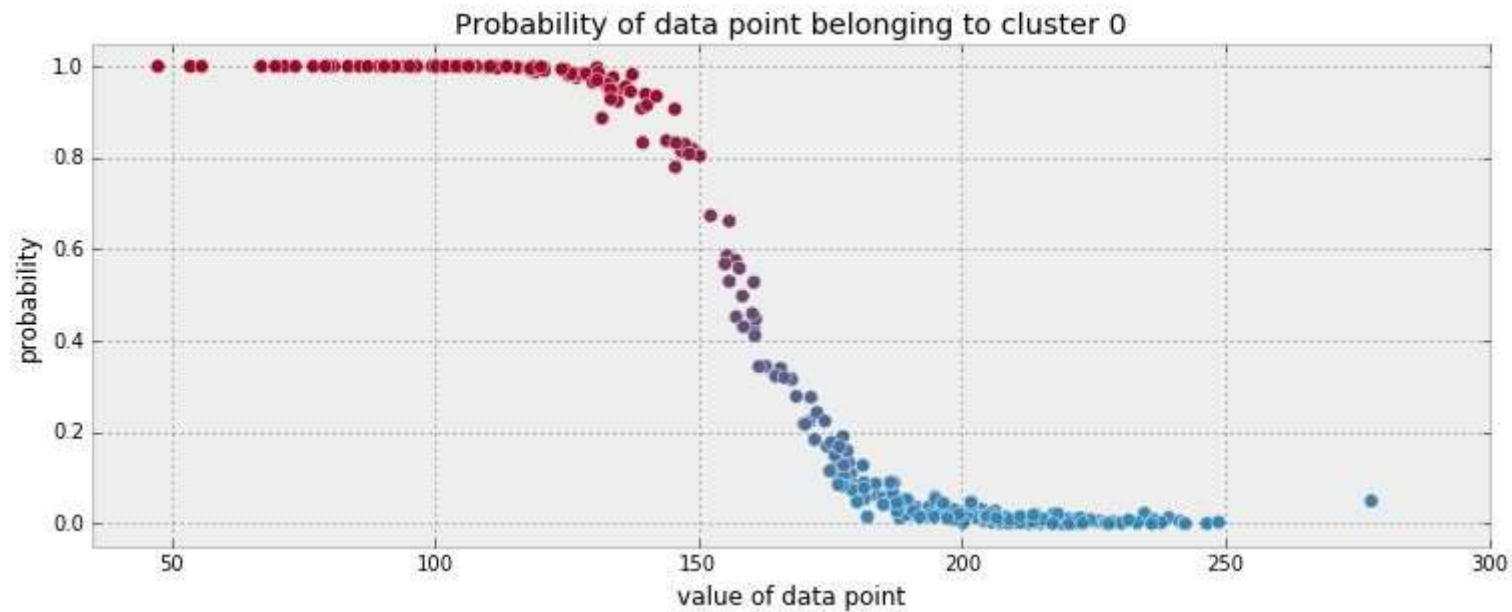
```
az.plot_trace(trace2.posterior.centers, figsize=(20, 4))
az.plot_trace(trace2.posterior.sds, figsize=(20, 4))
az.plot_trace(trace2.posterior.p, figsize=(20, 4))
plt.show()
```



Posterior Distributions of the Center and Standard Deviation Variables

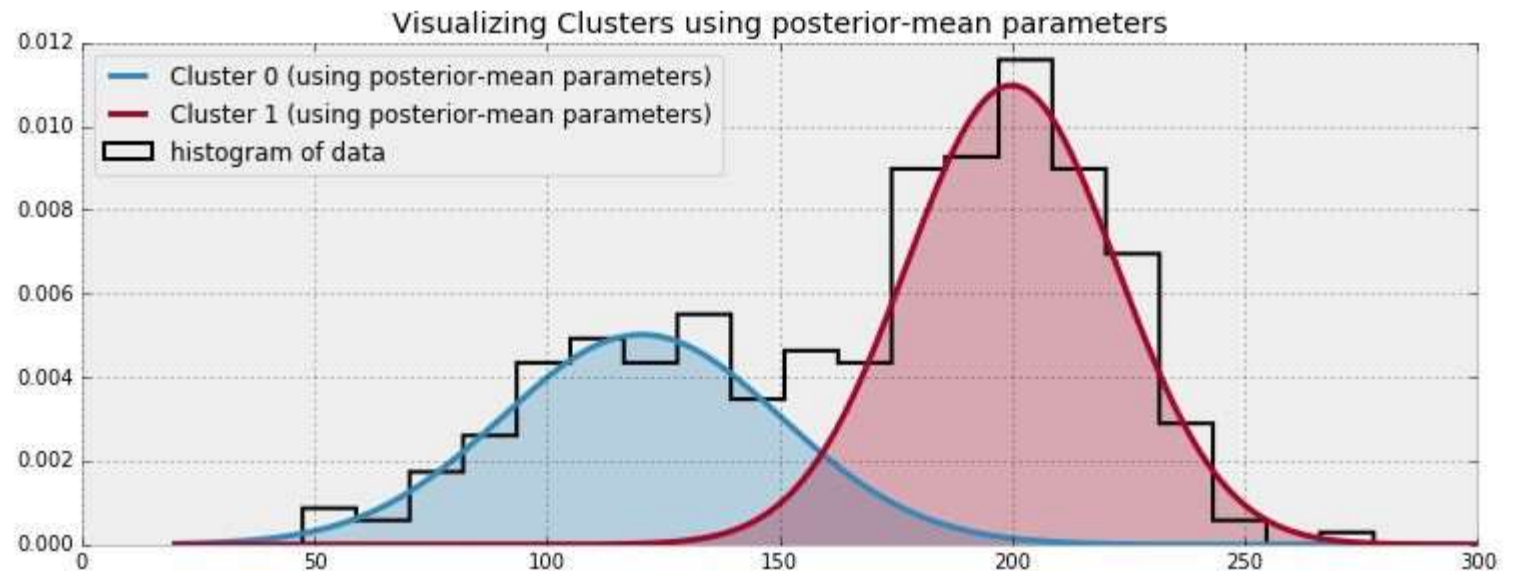


The Posterior Distributions for the Labels of the Data Points



How can we choose just a single pair of values for the mean and variance and determine a sorta-best-fit gaussian?

One quick way (which has nice theoretical properties), is to use the mean of the posterior distributions



Cluster Prediction

What about prediction? Suppose we observe a new data point, say $x = 175$, and we wish to label it to a cluster. It is foolish to simply assign it to the closer cluster center, as this ignores the standard deviation of the clusters, and we have seen from the plots above that this consideration is very important. More formally: we are interested in the probability (as we cannot be certain about labels) of assigning $x = 175$ to cluster 1. Denote the assignment of x as L_x , which is equal to 0 or 1, and we are interested in $P(L_x = 1|x = 175)$.

A naive method to compute this is to re-run the above MCMC with the additional data point appended. The disadvantage with this method is that it will be slow to infer for each novel data point. Alternatively, we can try a less precise, but much quicker method.

Cluster Prediction

For a particular sample set of parameters for our posterior distribution, $(\mu_0, \sigma_0, \mu_1, \sigma_1, p)$, we are interested in asking "Is the probability that x is in cluster 1 greater than the probability it is in cluster 0?", where the probability is dependent on the chosen parameters.

$$\begin{aligned} P(L_x = 1|x = 175) &> P(L_x = 0|x = 175) \\ \frac{P(x = 175|L_x = 1)P(L_x = 1)}{P(x = 175)} &> \frac{P(x = 175|L_x = 0)P(L_x = 0)}{P(x = 175)} \\ P(x = 175|L_x = 1)P(L_x = 1) &> P(x = 175|L_x = 0)P(L_x = 0) \end{aligned}$$

```
norm_pdf = stats.norm.pdf
```

```
x = 175
```

```
v = p_trace * norm_pdf(x, loc=center_trace[:, 0], scale=std_trace[:, 0]) > \
    (1 - p_trace) * norm_pdf(x, loc=center_trace[:, 1], scale=std_trace[:, 1])
```

```
print("Probability of belonging to cluster 0:", v.mean())    # Probability of belonging to cluster 0: 0.025
```


Diagnosing Convergence

MCMC: Science & Art

- Science:

The Markov chain will converge to the true posterior distribution... after infinite iterations

- Art:

To determine the minimum number of samples required to ensure a reasonable approximation to the target posterior density

Assessing Convergence

- One approach to analyzing convergence is analytical, whereby the variance of the sample at different sections of the chain are compared to that of the limiting distribution. These methods use distance metrics to analyze convergence, or place theoretical bounds on the sample variance, and though they are promising, they are generally difficult to use and are not prominent in the MCMC literature.
- More common is a statistical approach to assessing convergence. With this approach, rather than considering the properties of the theoretical target distribution, only the statistical properties of the observed chain are analyzed. Reliance on the sample alone restricts such convergence criteria to heuristics. As a result, convergence cannot be guaranteed. Although evidence for lack of convergence using statistical convergence diagnostics will correctly imply lack of convergence in the chain, the absence of such evidence will not guarantee convergence in the chain. Nevertheless, negative results for one or more criteria may provide some measure of assurance to users that their sample will provide valid inferences.
- **The most straightforward approach for assessing convergence is based on simply plotting and inspecting traces and histograms of the observed MCMC sample.**

Assessing Convergence: Good Signs

- If the trace of values for each of the stochastics exhibits asymptotic behavior over the last m iterations, this may be satisfactory evidence for convergence.
- A similar approach involves plotting a histogram for every set of k iterations (perhaps 50-100) beyond some burn in threshold n ; if the histograms are not visibly different among the sample intervals, this is reasonable evidence for convergence.
- The traces of several MCMC chains initialized with different starting values give similar results. Overlaying these traces on the same set of axes should (if convergence has occurred) show each chain tending toward the same equilibrium value, with approximately the same variance.
- Chain “mixes well”. (i.e., chain has run much longer than any observed timescale for correlation between samples)

Assessing Convergence: Warning Signs

- Differences within or across Markov chains
- “Poor mixing”
- Autocorrelation between states of Markov chain

Assessing Convergence: More Formal Methods

- Geweke: Compares the mean and variance of segments from the beginning and end of a single chain
- Raftery: Estimates the minimum chain length needed to estimate a percentile to some precision
- The Gelman-Rubin statistic uses an analysis of variance approach to assessing convergence. This diagnostic uses multiple chains to check for lack of convergence, and is based on the notion that if multiple chains have converged, by definition they should appear very similar to one another; if not, one or more of the chains has failed to converge

Autocorrelation

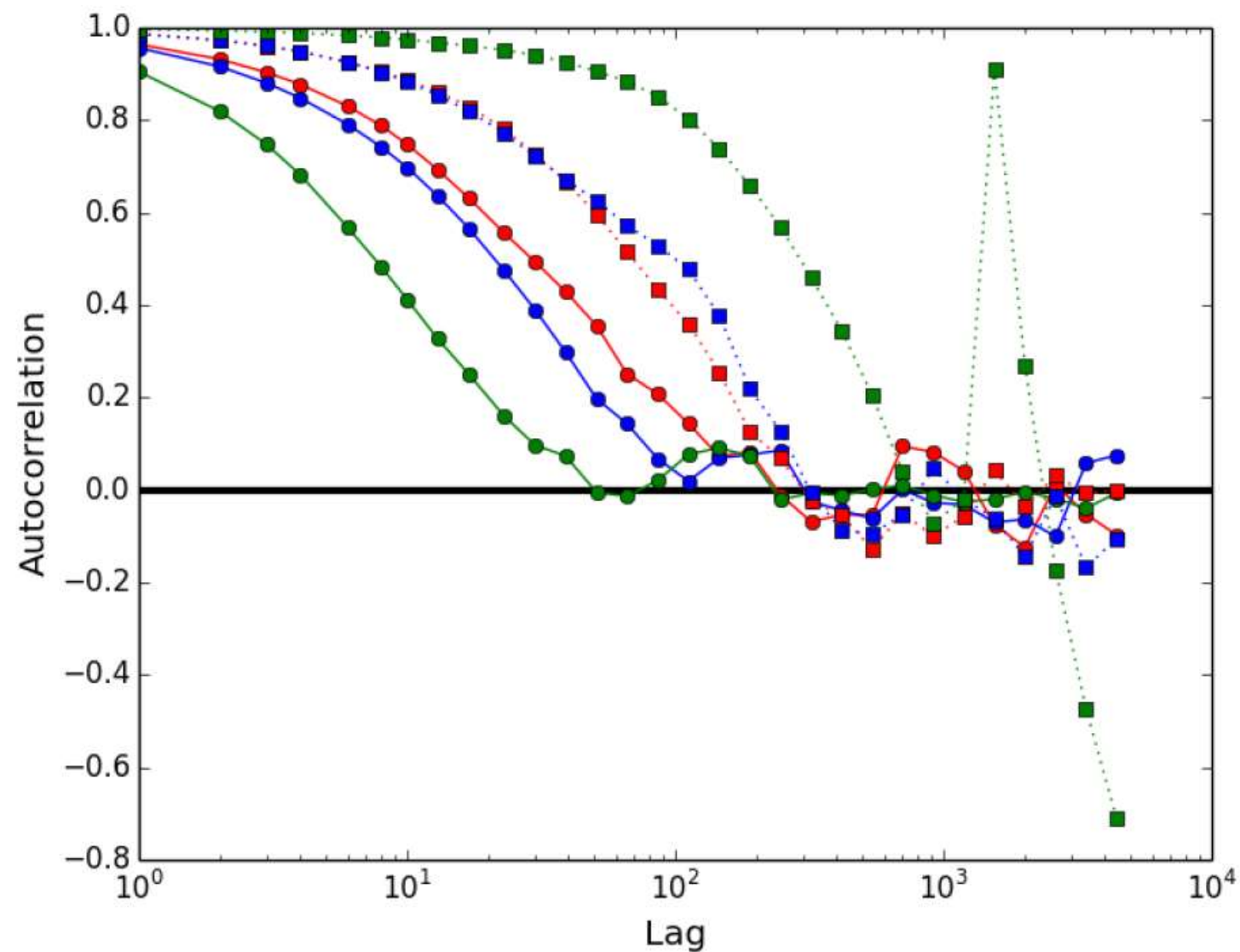
- Autocorrelation is a measure of how related a series of numbers is with itself. A measurement of 1.0 is perfect positive autocorrelation, 0 no autocorrelation, and -1 is perfect negative correlation. One way to think of autocorrelation is "If I know the position of the series at time s , can it help me know where I am at time t ?"
- Samples from MCMC algorithms are usually autocorrelated, due partly to the inherent Markovian dependence structure. The degree of autocorrelation can be quantified using the autocorrelation function:

$$\rho_k = \frac{\text{Cov}(X_t, X_{t+k})}{\sqrt{\text{Var}(X_t)\text{Var}(X_{t+k})}} = \frac{E[(X_t - \theta)(X_{t+k} - \theta)]}{\sqrt{E[(X_t - \theta)^2]E[(X_{t+k} - \theta)^2]}}$$

- A chain that is not exploring the space well will exhibit very high autocorrelation. Visually, if the trace seems to meander like a river, and not settle down, the chain will have high autocorrelation.

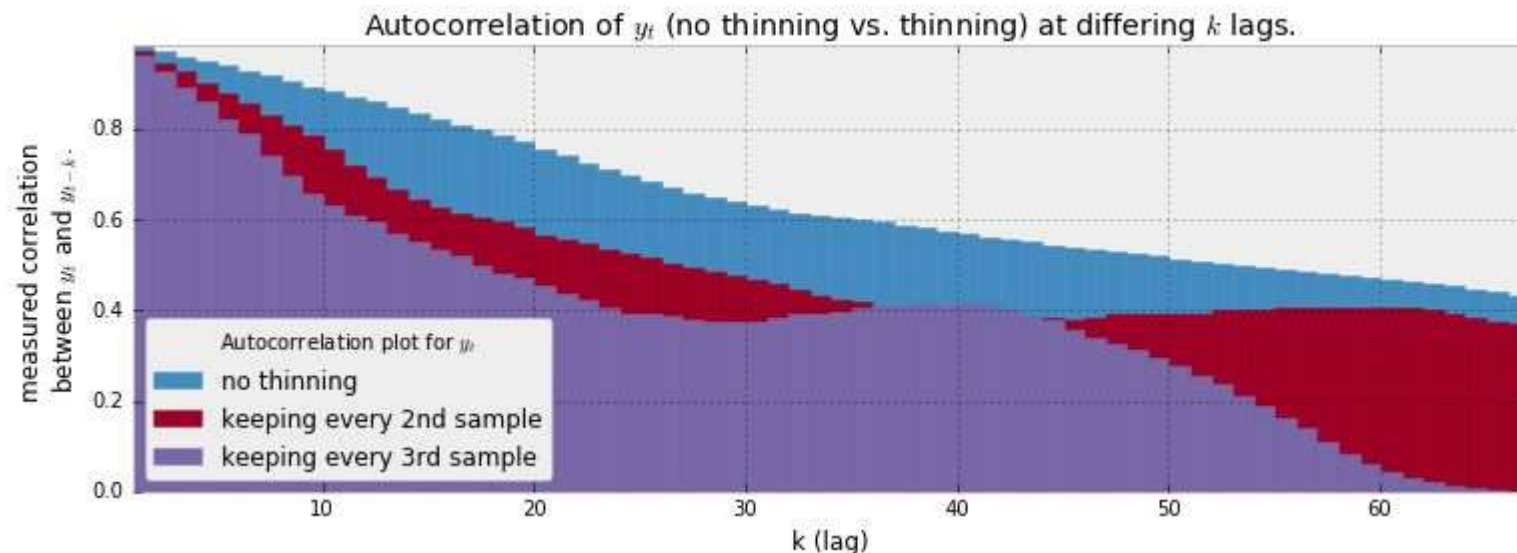
Autocorrelation

- What is smallest k (lag) to give an $\rho_k \approx 0$?
- One of several methods for estimating how many iterations of Markov chain are needed



Thinning

Another issue can arise if there is high-autocorrelation between posterior samples. Many post-processing algorithms require samples to be independent of each other. This can be solved, or at least reduced, by only returning to the user every n th sample, thus removing some autocorrelation. With more thinning, the autocorrelation drops quicker. There is a tradeoff though: higher thinning requires more MCMC iterations to achieve the same number of returned samples. For example, 10 000 samples unthinned is 100 000 with a thinning of 10 (though the latter has less autocorrelation).



Useful Tips for MCMC

Good heuristics to help convergence and speed up the MCMC

Intelligent Starting Values

- It would be great to start the MCMC algorithm off near the posterior distribution, so that it will take little time to start sampling correctly. We can aid the algorithm by telling where we *think* the posterior distribution will be by specifying the `value` parameter in the `stochastic` variable creation. In many cases we can produce a reasonable guess for the parameter. For example, if we have data from a Normal distribution, and we wish to estimate the μ parameter, then a good starting value would be the *mean* of the data.
- For most parameters in models, there is a frequentist estimate of it. These estimates are a good starting value for MCMC algorithms.

Using MAP to Improve Convergence

- Poor starting values can prevent any convergence, or significantly slow it down. Ideally, we would like to have the chain start at the *peak* of our landscape, as this is exactly where the posterior distributions exist. Hence, if we started at the "peak", we could avoid a lengthy burn-in period and incorrect inference. Generally, we call this "peak" the *maximum a posterior* or, more simply, the MAP.
- Of course, we do not know where the MAP is. PyMC provides a function that will approximate, if not find, the MAP location. In the PyMC main namespace is the `find_MAP` function. If you call this function within the context of `Model()`, it will calculate the MAP which you can then pass to `pm.sample()` as a start parameter.

```
with pm.Model() as model:
    start = pm.find_MAP()
    step = pm.Metropolis()
    trace = pm.sample(100000, step=step, initvals=start, tune=50000)
```

- The MAP can also be used as a solution to the inference problem, as mathematically it is the most likely value for the unknowns. But, this location ignores the uncertainty and doesn't return a distribution.

Priors

- If the priors are poorly chosen, the MCMC algorithm may not converge, or at least have difficulty converging. Consider what may happen if the prior chosen does not even contain the true parameter: the prior assigns 0 probability to the unknown, hence the posterior will assign 0 probability as well. This can cause pathological results.
- Often, lack of convergence or evidence of samples crowding to boundaries implies something is wrong with the chosen priors.

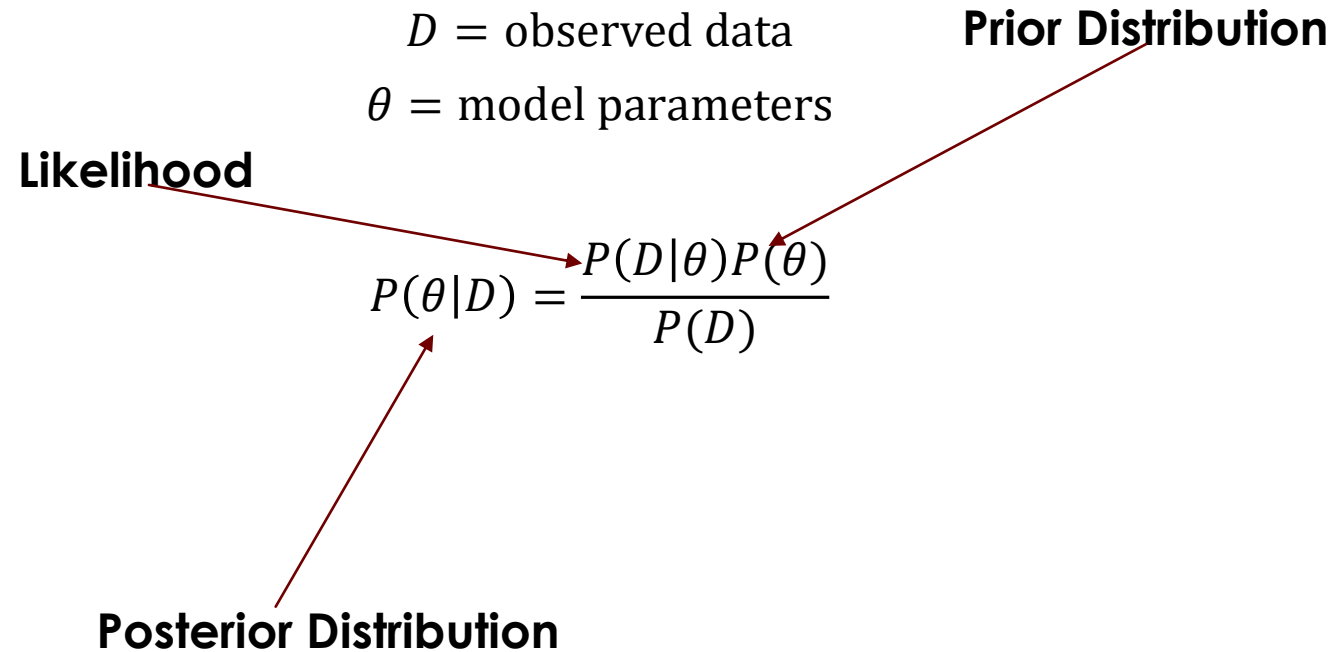
The Folk Theorem of Statistical Computing

If you are having computational problems, probably your model is wrong

Approximate Bayesian Computation

Life Beyond Likelihood

From Bayes theorem, to compute a posterior we need two basic ingredients, a prior and a likelihood. However, for particular problems, we may find that we can not express the likelihood in closed-form, or it is prohibitively costly to compute it. This seems to be a dead end for our Bayesian approach. But that is not necessarily the case as long as we are able to somehow generate synthetic data. This generator of synthetic data is generally referred to as a *simulator*. From the perspective of the ABC method the simulator is a black-box, we feed parameter values at one side and get simulated data from the other. The complication we add however, is uncertainty about which inputs are good enough to generate synthetic data similar to the observed data.



Approximate Bayesian Computation (ABC)

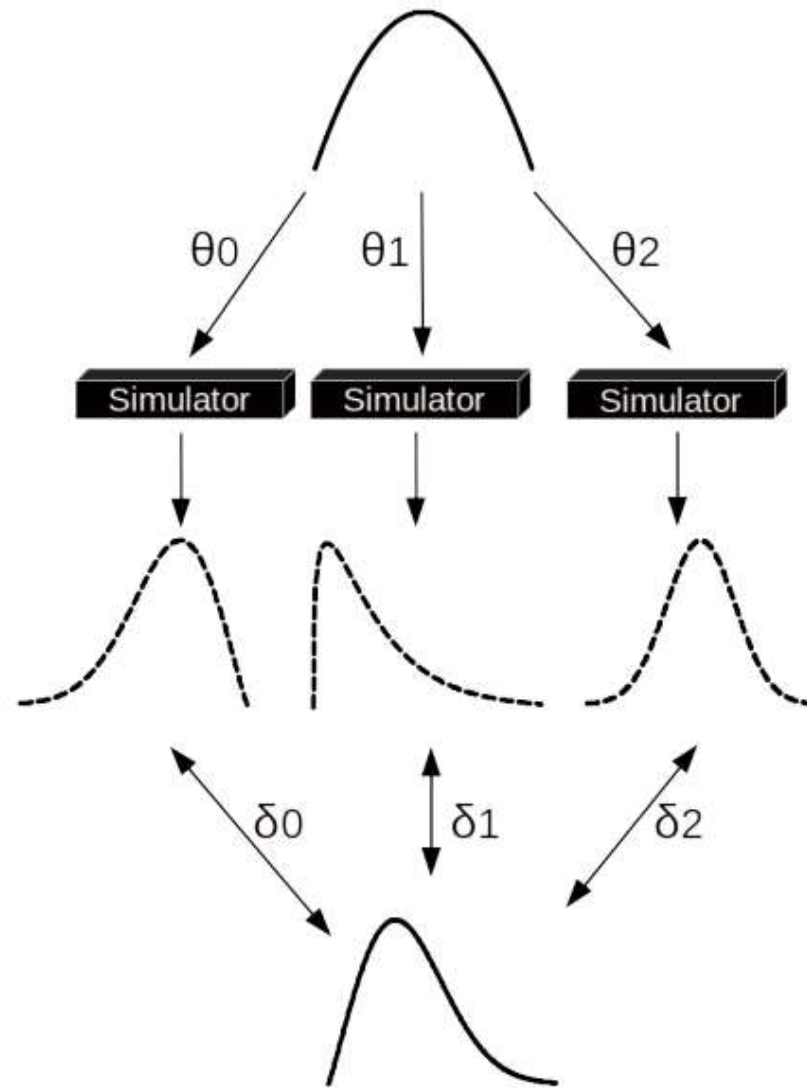
Approximate Bayesian Computation methods (also called likelihood free inference methods), are a group of techniques developed for inferring posterior distributions in cases where the likelihood function is intractable or costly to evaluate. This does not mean that the likelihood function is not part of the analysis, it just that we are approximating the likelihood, and hence the name of the ABC methods.

ABC comes useful when modeling complex phenomena in certain fields of study, like systems biology. Such models often contain unobservable random quantities, which make the likelihood function hard to specify, but data can be simulated from the model.

Approximating the Approximated Posterior

These methods follow a general form:

1. Sample a value of θ^* from the prior /proposal distribution $\pi(\theta)$.
2. Pass that value to the simulator and generate synthetic data, i.e. simulate a data set y^* using a function that takes θ and returns a data set of the same dimensions as the observed data set y_0 .
3. Compare the simulated dataset y^* with the experimental data set y_0 using a distance function δ and a tolerance threshold ε . If the synthetic data is at a distance closer than ε save the proposed θ , otherwise reject it. In some cases a distance function is computed between two summary statistics $\delta(S(y_0), S(y^*))$, avoiding the issue of computing distances for entire datasets.
4. Repeat until having the desired number of samples.



One step of an ABC-rejection sampler. We sample a set of θ values from the prior distribution (at the top). Each value is passed to the simulator, which generates synthetic datasets (dashed distributions), we compare the synthetic data with the observed one (the distribution at the bottom). In this example only θ_1 was able to generate a synthetic dataset close enough to the observed data, thus θ_0 and θ_2 were rejected. Notice that if we were using a summary statistics, instead of the entire dataset we should compute the summary statistics of the synthetic and observed data after step 2 and before step 3.

Sequential Monte Carlo (SMC)

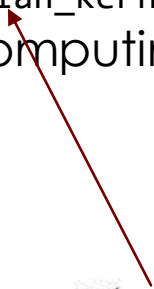
The major drawback of the ABC-rejection sampler is that if the prior distribution is too different from the posterior distribution we will spend most of the time proposing values that will be rejected. A better idea is to propose from a distribution closer to the actual posterior. Generally we do not know enough about the posterior to do this by hand, but we can achieve it using a Sequential Monte Carlo (SMC) method. This is a general sampler method, like the MCMC methods. SMC can be adapted to perform ABC and then it is called SMC-ABC. SMC proceeds by increasing the value of an auxiliary parameter β in s successive stages $\{\beta_0 = 0 < \beta_1 < \dots < \beta_s = 1\}$. This is done in such a way that we start sampling from the prior ($\beta = 0$) until we reach the posterior ($\beta = 1$). Thus, we can think of β as a parameter that *gradually turns the likelihood on*. The intermediate values of β are automatically computed by SMC.

Fitting a Gaussian the ABC-way

To illustrate how to use ABC, we are going to start with a very simple example estimating the mean and standard deviation of Gaussian data. Clearly under normal circumstances using a Gaussian likelihood will do the job very well. But that would defeat the purpose of this example. So, instead of that we are going to define a simulator. A very straightforward simulator for normal data is a pseudo random number generator, in real life our simulator will be most likely something fancier.

The ABC Model in PyMC

Defining an ABC model in PyMC is in general, very similar to defining other PyMC models. The two important differences are: we need to define a `simulator` distribution and we need to use `sample_smc`. The `simulator` works as a generic interface to pass the synthetic data generating function (`normal_sim` in this example), its parameters, the observed data and optionally a distance function and a summary statistics. In the following code we are using the default distance, `gaussian_kernel`, and the `sort` summary_statistic. As the name suggests `sort` sorts the data before computing the distance.


$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right)$$

The ABC Model in PyMC

```
data = np.random.normal(0, 1, 1000)

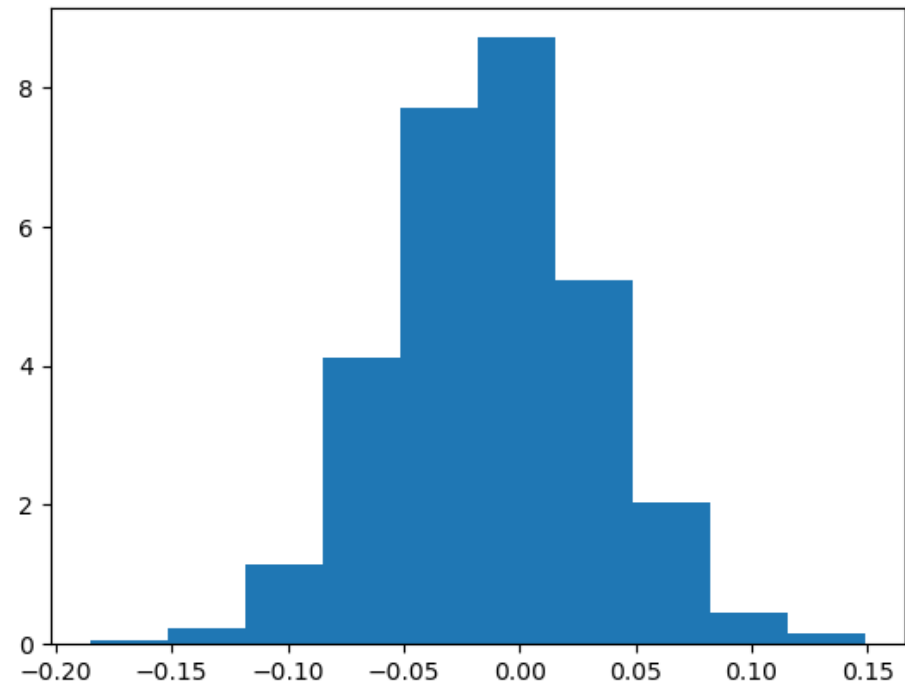
def normal_sim(rng, a, b, size=1000):
    return rng.normal(a, b, size=size)

with pm.Model() as abc_model:
    a = pm.Normal("a", mu=0, sigma=5)
    b = pm.HalfNormal("b", sigma=1)
    s = pm.Simulator("s", normal_sim, params=(a, b), sum_stat="sort", epsilon=1, observed=data)

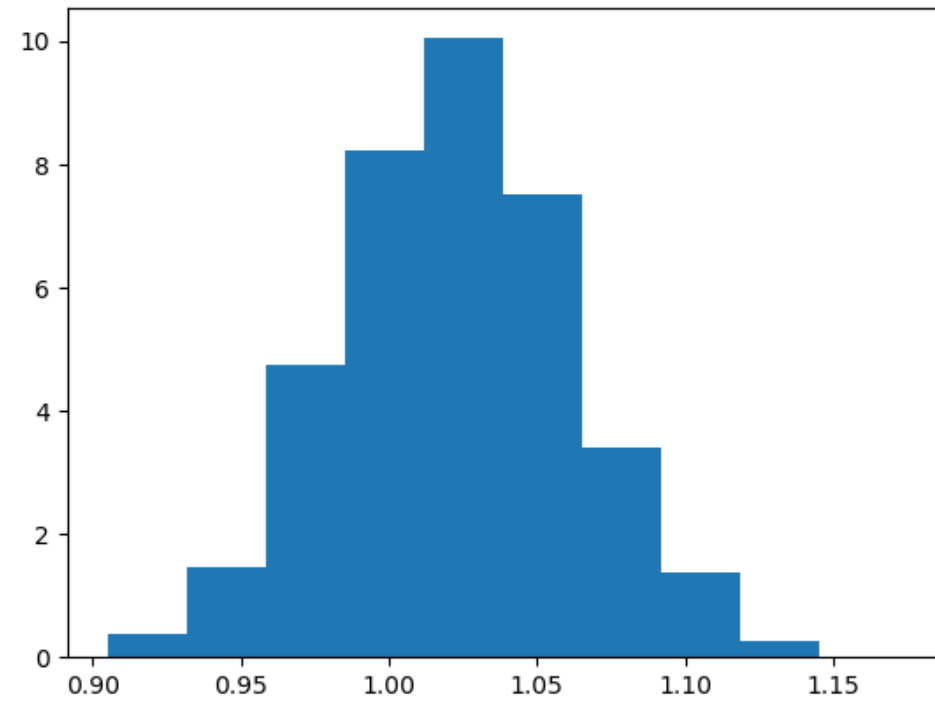
    trace = pm.sample_smc(chains=1, return_inferencedata=False)

a_samples = trace['a']
b_samples = trace['b']
```

The Results



Alpha



Beta